Монгол улсын их сургууль
Мэдээллийн технологи, электроникийн сургууль

Мэдээлэл , компьютерийн ухааны тэнхим

Семинарын даалгавар №5
OpenAPI

Гүйцэтгэсэн:

Програм хангамж-4

Э.Түвшин-Эрдэнэ (23b1num0869)

Улаанбаатар хот

2026 он

Machine-readable OpenAPI 3.0 Specification

```yaml
openapi: 3.0.3
info:
 title: Corg.ly Pet Onboarding & Translation API
 version: 1.0.0
 description: Official API documentation for pet registration, voice
translation, and webhooks.
servers:
 - url: https://api.corg.ly/v1
   description: Production Server

paths:
 /pets:
   get:
     summary: List all registered pets
     operationId: listPets
     responses:
       '200':
         description: A list of pets
         content:
           application/json:
             example:
               - pet_id: "corgi_98231"
                 pet_name: "Einstein"
                 breed: "Pembroke Welsh Corgi"
                 photo_url:
"https://media.corg.ly/photos/einstein.jpg"
             schema:
               type: array
               items:
                 $ref: '#/components/schemas/PetProfile'
       '500':
         $ref: '#/components/responses/ServerError'

 /pets/{pet_id}:
   get:
     summary: Retrieve specific pet details
     operationId: getPetById
```

```yaml
    parameters:
      - name: pet_id
        in: path
        required: true
        schema:
          type: string
        example: "corgi_98231"
    responses:
      '200':
        description: Pet details found
        content:
          application/json:
            example:
              pet_id: "corgi_98231"
              pet_name: "Einstein"
              breed: "Pembroke Welsh Corgi"
              photo_url: "https://media.corg.ly/photos/einstein.jpg"
            schema:
              $ref: '#/components/schemas/PetProfile'
      '404':
        $ref: '#/components/responses/NotFoundError'
      '500':
        $ref: '#/components/responses/ServerError'

/pets/upload-photo:
  post:
    summary: Upload a pet profile photo
    operationId: uploadPetPhoto
    security:
      - BearerAuth: []
    requestBody:
      required: true
      content:
        multipart/form-data:
          schema:
            type: object
            required: [pet_id, photo]
            properties:
              pet_id:
```

```
                type: string
                example: "corgi_98231"
              photo:
                type: string
                format: binary
    responses:
      '200':
        description: Photo uploaded successfully
        content:
          application/json:
            example:
              status: "success"
              photo_url: "https://media.corg.ly/photos/einstein.jpg"
      '400':
        $ref: '#/components/responses/BadRequestError'
      '401':
        $ref: '#/components/responses/UnauthorizedError'
      '500':
        $ref: '#/components/responses/ServerError'

/audio/translate-bark:
  post:
    summary: Translate pet bark audio to English text
    operationId: translateBark
    security:
      - BearerAuth: []
    requestBody:
      required: true
      content:
        multipart/form-data:
          schema:
            type: object
            required: [audio_file]
            properties:
              audio_file:
                type: string
                format: binary
    responses:
      '200':
```

```yaml
        description: Audio processed successfully
        content:
          application/json:
            example:
              translation: "I demand treats immediately!"
              confidence: 0.98
      '400':
        $ref: '#/components/responses/BadRequestError'
      '401':
        $ref: '#/components/responses/UnauthorizedError'
      '500':
        $ref: '#/components/responses/ServerError'

/webhooks/subscribe:
  post:
    summary: Subscribe to pet activity webhooks
    operationId: subscribeWebhook
    security:
      - BearerAuth: []
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            required: [target_url, event_type]
            properties:
              target_url:
                type: string
                example: "https://myapp.com/webhooks/corgly"
              event_type:
                type: string
                example: "pet.bark_detected"
    responses:
      '201':
        description: Webhook subscription created
        content:
          application/json:
            example:
```

```
                subscription_id: "sub_88321"
                status: "active"
        '400':
          $ref: '#/components/responses/BadRequestError'
        '401':
          $ref: '#/components/responses/UnauthorizedError'
        '500':
          $ref: '#/components/responses/ServerError'

components:
 securitySchemes:
   BearerAuth:
     type: http
     scheme: bearer
     bearerFormat: JWT

 schemas:
   PetProfile:
     type: object
     required: [pet_id, pet_name, breed]
     properties:
       pet_id:
         type: string
         example: "corgi_98231"
       pet_name:
         type: string
         example: "Einstein"
       breed:
         type: string
         example: "Pembroke Welsh Corgi"
       photo_url:
         type: string
         example: "https://media.corg.ly/photos/einstein.jpg"

 responses:
   BadRequestError:
     description: Invalid request parameters or payload
     content:
       application/json:
```

```
        example:
          error: "Bad Request"
          message: "Missing required form-data field"
    UnauthorizedError:
      description: Missing or invalid JWT auth token
      content:
        application/json:
          example:
            error: "Unauthorized"
            message: "Bearer token invalid"
    NotFoundError:
      description: Requested resource was not found
      content:
        application/json:
          example:
            error: "Not Found"
            message: "Resource does not exist"
    ServerError:
      description: Internal server error
      content:
        application/json:
          example:
            error: "Internal Server Error"
            message: "An unexpected error occurred"
```

Python Кодны жишээ ба чанарын аудит

Bhatti-ийн 5 зарчмаар кодны чанарыг үнэлэх үүднээс дараах жишээ кодыг бичив.

```
import requests

#
================================================================================
# Endpoint 1: POST /v1/pets/upload-photo
# Context: Uploads a local JPEG image file to update a specific pet's profile
picture.
# Assumes an active API JWT token and a valid local file path.
#
================================================================================
auth_token = "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
headers = {"Authorization": f"Bearer {auth_token}"}

with open("einstein.jpg", "rb") as image_file:
   payload_files = {"photo": image_file}
   payload_data = {"pet_id": "corgi_98231"}

   upload_response = requests.post(
       "https://api.corg.ly/v1/pets/upload-photo",
       headers=headers,
       data=payload_data,
       files=payload_files
   )

print(upload_response.json())
# Expected Output: {"status": "success", "photo_url":
"https://media.corg.ly/photos/einstein.jpg"}


#
================================================================================
# Endpoint 2: POST /v1/audio/translate-bark
# Context: Sends a pet audio recording to the translation engine to obtain a
human-language meaning.
# Assumes a valid audio file named 'sample_bark.wav' in the execution path.
#
================================================================================
with open("sample_bark.wav", "rb") as audio_file:
   payload_files = {"audio_file": audio_file}

   translation_response = requests.post(
```

```
        "https://api.corg.ly/v1/audio/translate-bark",
        headers=headers,
        files=payload_files
    )

print(translation_response.json())
# Expected Output: {"translation": "I demand treats immediately!",
"confidence": 0.98}



#
================================================================================
# Endpoint 3: POST /v1/webhooks/subscribe
# Context: Registers a customer webhook endpoint to receive notifications when
a bark is processed.
# Assumes a publicly accessible HTTPS endpoint ready to accept HTTP POST calls.
#
================================================================================
subscription_payload = {
    "target_url": "https://myapp.com/webhooks/corgly",
    "event_type": "pet.bark_detected"
}

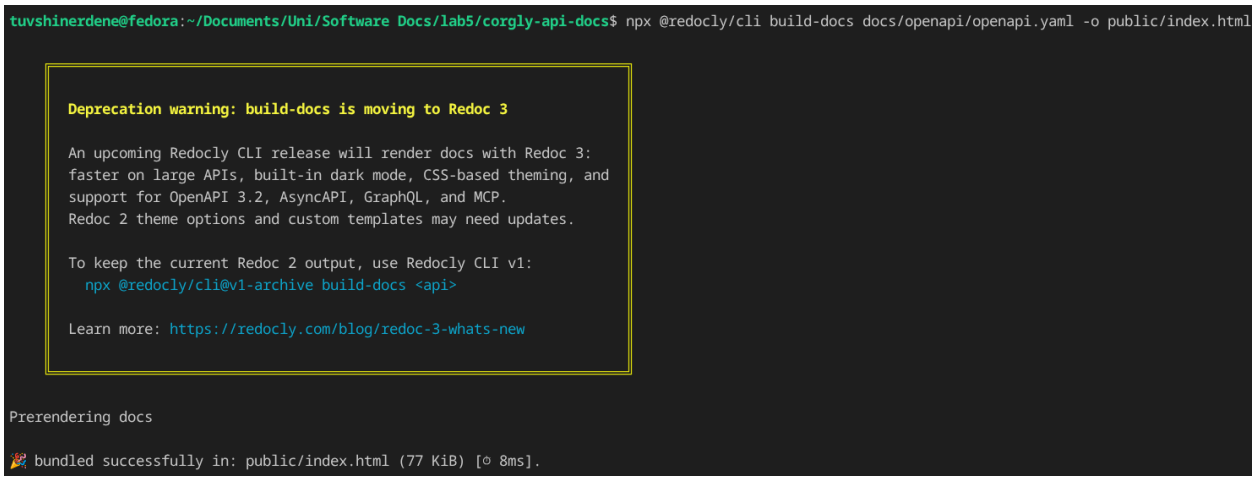
webhook_response = requests.post(
    "https://api.corg.ly/v1/webhooks/subscribe",
    headers=headers,
    json=subscription_payload
)

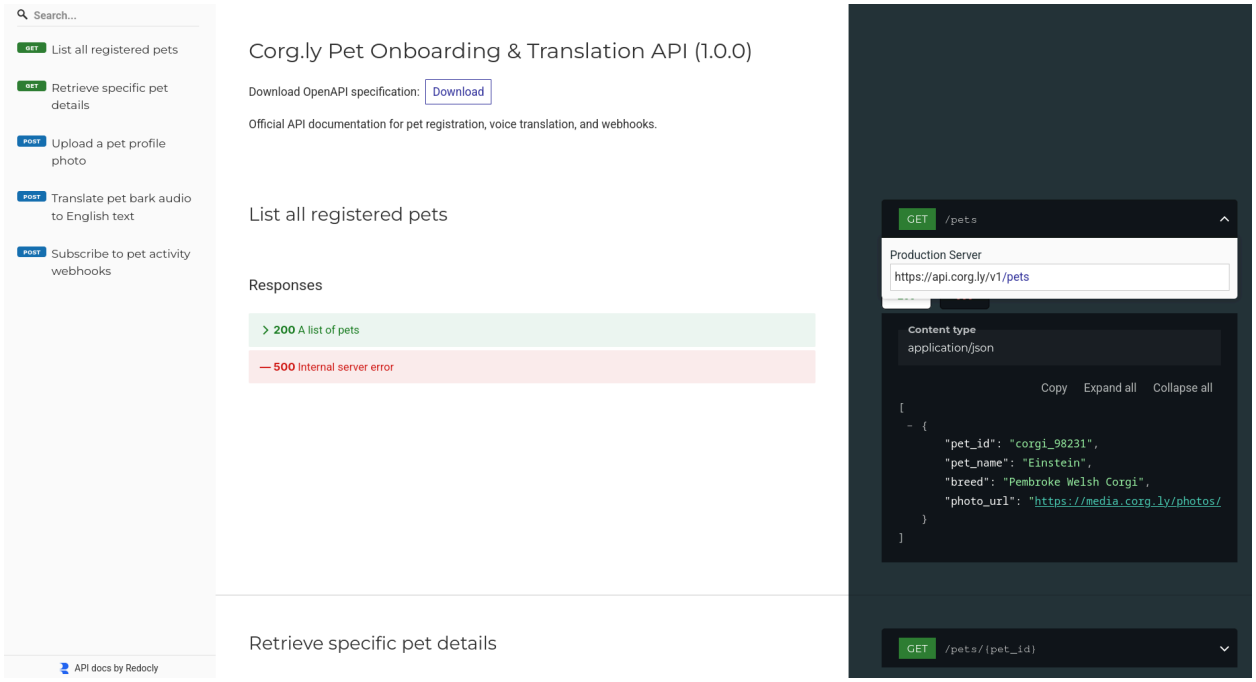
print(webhook_response.json())
# Expected Output: {"subscription_id": "sub_88321", "status": "active"}
```

Sample code target	Explained (1-5)	Concise (1-5)	Clear (1-5)
POST /v1/pets/upload-photo	5	4	5
POST /v1/audio/translate-bark	4	5	5
POST /v1/webhooks/subscribe	5	4	5

Dual Renderer Hosting & Comparative Selection Report

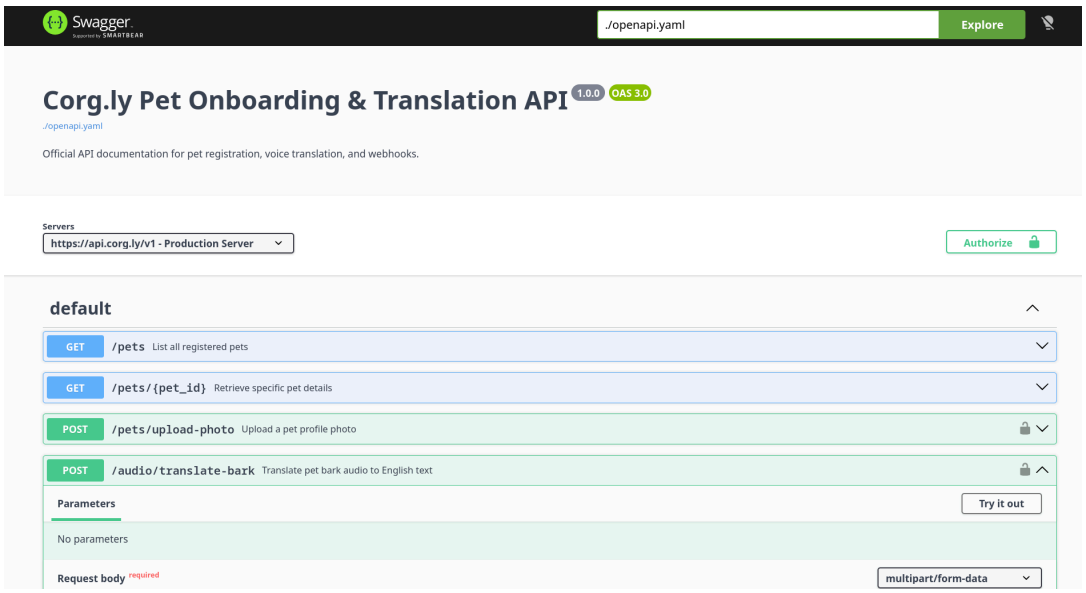
Дараах коммандаар Redoc HTML бичиг баримтыг үүсгэв:





Дараах коммандаар Swagger UI -ийг локал орчинд ажиллуулав.

```
tuvshinerdene@fedora:~/Documents/Uni/Software Docs/lab5/corgly-api-docs$ docker run -d -p 8080:8080 -e SWAGGER_JSON=/spec/openapi.yaml -v "$(pwd)/d
cs/openapi":/spec:z swaggerapi/swagger-ui
Unable to find image 'swaggerapi/swagger-ui:latest' locally
latest: Pulling from swaggerapi/swagger-ui
f519988c5264: Pull complete
af7dd138f459: Pull complete
645150f6ad24: Pull complete
223a1665b163: Pull complete
807d6f93a782: Pull complete
2fd0eeb63850: Pull complete
51900e10fb9c: Pull complete
bc98d7675616: Pull complete
861f39ed5156: Pull complete
8f924cf5086c: Pull complete
58c524ea09ce: Pull complete
55afa1ecc21d: Pull complete
850bf2dcecff: Pull complete
```



Redoc-ийн 3 баганатай хэлбэр нь уншихад илүү хялбар санагдсан ба client-side хайлт нь төвөгтэй endpoint-уудын параметр болон бусад хэрэгтэй мэдээллийг олоход хялбар болгож байна.

Харин Swagger UI нь шууд endpoint-уудыг туршин үзэх боломжтой боловч 1 баганатай харагдац нь том хэмжээний API- ийг ойлгоход төвөгтэй болгож байна.

Албан ёсны бичиг баримтад Redoc-ийг, харин endpoint тестлэлд Swagger UI илүү тохиромжтой санагдав.

Reflective questions

1. **Which concept from Bhatti Ch.5 challenged your previous coding/documentation habits the most ?**
   The requirement for Trustworthy Payloads challenged my previous habit of writing mock JSON responses. Ensuring that code samples match live server outputs prevents copy-paste errors for downstream API consumers.

2. **Which concept from Chinchilla will you apply in future professional projects?**
   Applying standardized API contract design and machine - readable schema validation to catch breaking changes early in CI pipelines.

3. **Where in your guiding project is the biggest gap between “what literature teaches” vs “real implementation”?**
   Literature assumes end to end sandbox testing servers exist for all endpoints. However, real implementation often requires handling dynamic media uploads that need dedicated server setups before code samples can run reliably.